MATLAB CODES SHORTENED:

% EXP 1: Large-scale fading

```
clc; clear; close all; rng(1);
fc=1.5e9; d=(1:2:31).^2; d0=100; sigma=3;
Gt=[1 1 .5]; Gr=[1 .5 .5]; n=[2 3 6];
P0=20*log10(4*pi*d0*fc/3e8);
for k=1:3
   F(k,:)=20*log10(4*pi*d*fc/3e8)-10*log10(Gt(k)*Gr(k));
   D(k,:)=P0+10*n(k)*log10(d/d0);
   S(k,:)=P0+20*log10(d/d0)+sigma*randn(size(d));
end
figure; semilogx(d,F(1,:),'r-o',d,F(2,:),'r-^',d,F(3,:),'r-s');
title('Free Path-loss Model, f_c=1500MHz');
legend('G_t=1, G_r=1','G_t=1, G_r=0.5','G_t=0.5, G_r=0.5');
axis([1 1000 40 110]); xlabel('Distance[m]'); ylabel('Path loss[dB]'); grid on;
figure; semilogx(d,D(1,:),'r-o',d,D(2,:),'r-^',d,D(3,:),'r-s');
title('Log-distance Path-loss Model, f_c=1500MHz');
legend('n=2','n=3','n=6');
axis([1 1000 40 110]); xlabel('Distance[m]'); ylabel('Path loss[dB]'); grid on;
figure; semilogx(d,S(1,:),'r-o',d,S(2,:),'r-^',d,S(3,:),'r-s');
title('Log-normal Path-loss Model, f_c=1500MHz, sigma=3dB, n=2');
legend('path 1','path 2','path 3');
axis([1 1000 40 110]); xlabel('Distance[m]'); ylabel('Path loss[dB]'); grid on;
```

EXP-2 SHORT CODE DIVERSITY

```
clc; clear; close all; rng(1);
SNR=-10:2:12; N=200000; B=zeros(numel(SNR),4,2,3);
for type=1:2 % 1=Rayleigh, 2=Rician
  for L=1:4 % Diversity branches
    for j=1:numel(SNR)
      bits=randi([0 1],1,2*N);
      s=(2*bits(1:N)-1)+1i*(2*bits(N+1:end)-1);
      h=(randn(L,N)+1i*randn(L,N))/sqrt(2);
      if type==2, h=(sqrt(1.5)+h)/sqrt(2.5); end
      r=h.*s+(randn(L,N)+1i*randn(L,N))/sqrt(2*10^(SNR(j)/10));
      [~,ix]=max(abs(h).^2,[],1);
      id=sub2ind([L N],ix,1:N);
      z{1}=conj(h(id)).*r(id); % SC
      z{2}=sum(conj(h).*r,1); % MRC
      z{3}=sum(conj(h)./abs(h).*r,1); % EGC
      for m=1:3
        B(j,L,type,m)=mean([real(z{m})>0 imag(z{m})>0]~=bits);
      end
    end
  end
end
names={'Selective Combining','Maximal Ratio Combining','Equal Gain Combining'};
channel={'Rayleigh','Ricean'};
for type=1:2
  for m=1:3
    figure; semilogy(SNR,squeeze(B(:,:,type,m)),'-*'); grid on;
    title(['BER of ' names{m} ' (' channel{type} ')']);
    xlabel('SNR (dB)'); ylabel('Bit error rate');
    legend('L=1','L=2','L=3','L=4');
  end
end
for type=1:2
  figure;
  semilogy(SNR,B(:,4,type,1),'-+',SNR,B(:,4,type,3),'-*',SNR,B(:,4,type,2),'-o');
  grid on; title(['BER of SC & MRC & EGC (' channel{type} ')']);
  xlabel('SNR (dB)'); ylabel('Bit error rate');
  legend('Selective Combining','Equal Gain Combining','Maximal Ratio Combining');
end
```

EXP 3 ALMOUTI SHORT CODE

```
% EXP 3: Alamouti 2x1 / 2x2 over a Rician channel
% Requires Communications Toolbox: pskmod, pskdemod, berfading, biterr
clc; clear; close all; rng(1);
Eb=0:10; Ks=[4 .6]; Ms=[4 8]; N=40000; % CHANGE THE K VALUE HERE ACCORDING TO
QUESTION
for ki=1:2
  K=Ks(ki); figure; hold on;
  for nr=1:2
    for mi=1:2
      M=Ms(mi); b=log2(M); sim=zeros(size(Eb));
      for j=1:numel(Eb)
        q=randi([0 M-1],2*N,1); s=pskmod(q,M,0,'gray');
        s1=s(1:2:end); s2=s(2:2:end);
        u=zeros(N,1); v=zeros(N,1);
        h=(sqrt(K)+(randn(nr,2,N)+1i*randn(nr,2,N))/sqrt(2))/sqrt(K+1);
        n0=1/(b*10^(Eb(j)/10));
        for r=1:nr
          h1=reshape(h(r,1,:),N,1); h2=reshape(h(r,2,:),N,1);
          y1=(h1.*s1+h2.*s2)/sqrt(2)+sqrt(n0/2)*(randn(N,1)+1i*randn(N,1));
          y2=(-h1.*conj(s2)+h2.*conj(s1))/sqrt(2)+sqrt(n0/2)*(randn(N,1)+1i*randn(N,1));
          u=u+conj(h1).*y1+h2.*conj(y2);
          v=v+conj(h2).*y1-h1.*conj(y2);
        end
        dec=pskdemod(reshape([u.';v.'],[],1),M,0,'gray');
        sim(j)=biterr(q,dec,b)/(2*N*b);
      end
      a=(nr-1)*2+mi;
      pSim(a)=semilogy(Eb,sim,'o-','LineWidth',1.1);
      pTh(a)=semilogy(Eb,berfading(Eb,'psk',M,2*nr,K),'-');
    end
  end
  set(gca,'YScale','log'); grid on;
  title(['Rician model (K = ' num2str(K) ')']);
  xlabel('EbNo (dB)'); ylabel('BER');
  legend([pSim(1:2) pTh(1:2) pSim(3:4) pTh(3:4)], ...
    'QPSK(simulated) 2x1','8-PSK(simulated) 2x1', ...
    'QPSK(theory) 2x1','8-PSK(theory) 2x1', ...
    'QPSK(simulated) 2x2','8-PSK(simulated) 2x2', ...
    'QPSK(theory) 2x2','8-PSK(theory) 2x2');
end
```

% EXP 4: 2x2 QPSK MIMO with ZF-SIC, MMSE-SIC and ML

```
% Requires Communications Toolbox: pskmod, pskdemod, biterr
clc; clear; close all; rng(1);
Eb=-10:5:20; N=10000; BER=zeros(numel(Eb),3);
Q=pskmod((0:3)',4);
[a,b]=ndgrid(1:4);
C=[Q(a(:)) Q(b(:))]';
for j=1:numel(Eb)
  n0=1/(2*10^(Eb(j)/10));
  for k=1:N
     idx=randi(4,2,1); s=reshape(Q(idx),2,1);
     H=(randn(2)+1i*randn(2))/sqrt(2);
     y=H*s+sqrt(n0/2)*(randn(2,1)+1i*randn(2,1));
     for m=1:2
        if m==1, W=pinv(H); else, W=(H'*H+n0*eye(2))\H'; end
        [~,first]=min(sum(abs(W).^2,2)); second=3-first;
        s1=pskmod(pskdemod(W(first,:)*y,4),4);
        rest=y-H(:,first)*s1;
        s2=(H(:,second)'*rest)/(H(:,second)'*H(:,second));
        est=zeros(2,1); est(first)=pskdemod(s1,4);
        est(second)=pskdemod(s2,4);
        BER(j,m)=BER(j,m)+biterr(idx-1,est,2);
     end
     [~,best]=min(sum(abs(y-H*C).^2,1));
     BER(j,3)=BER(j,3)+biterr(idx-1,pskdemod(C(:,best),4),2);
  end
end
BER=BER/(4*N);
figure; semilogy(Eb,BER(:,1),'r*-',Eb,BER(:,2),'bo-',Eb,BER(:,3),'gd-');
grid on; axis([-10 20 1e-3 1]);
title('2x2 Uncoded QPSK System');
xlabel('Eb/No (dB)'); ylabel('BER');
legend('ZF-SIC','MMSE-SIC','ML');
```

% EXP 5: Genuine 802.11n packet-error-rate experiment (not a drawn curve)

```
% Requires WLAN Toolbox and Communications Toolbox
clc; clear; close all; rng(1);
cfg=wlanHTConfig('ChannelBandwidth','CBW20','NumTransmitAntennas',2, ...
  'NumSpaceTimeStreams',2,'PSDULength',1000,'MCS',15,'ChannelCoding','BCC');
fs=wlanSampleRate(cfg); ind=wlanFieldIndices(cfg);
ofdm=wlanHTOFDMInfo('HT-Data',cfg);
ch=wlanTGnChannel('SampleRate',fs,'DelayProfile','Model-B', ...
  'NumTransmitAntennas',2,'NumReceiveAntennas',2, ...
  'TransmitReceiveDistance',10,'LargeScaleFadingEffect','None');
snr=10:5:50; PER=zeros(size(snr));
maxErrors=200; maxPackets=10000; % Same limits as the record
for j=1:numel(snr)
  eff=snr(j)-10*log10(ofdm.FFTLength/ofdm.NumTones);
  noise=comm.AWGNChannel('NoiseMethod','Signal to noise ratio (SNR)', ...
    'SNR',eff,'SignalPower',.5);
  nVar=.5*10^(-eff/10); errors=0; packets=0;
  while errors<maxErrors && packets<maxPackets
    bits=randi([0 1],8*cfg.PSDULength,1);
    tx=[wlanWaveformGenerator(bits,cfg);zeros(15,2)];
    reset(ch); rx=noise(ch(tx)); packets=packets+1;
    p=wlanPacketDetect(rx,cfg.ChannelBandwidth);
    if isempty(p) || p>15, errors=errors+1; continue; end
    lstf=rx(p+(ind.LSTF(1):ind.LSTF(2)),:);
    rx=frequencyOffset(rx,fs,-wlanCoarseCFOEstimate(lstf,cfg.ChannelBandwidth));
    p=p+wlanSymbolTimingEstimate(rx(p+(ind.LSTF(1):ind.LSIG(2)),:),cfg.ChannelBandwidth);
    if p<0 || p>15, errors=errors+1; continue; end
    lltf=rx(p+(ind.LLTF(1):ind.LLTF(2)),:);
    rx=frequencyOffset(rx,fs,-wlanFineCFOEstimate(lltf,cfg.ChannelBandwidth));
    ltf=rx(p+(ind.HTLTF(1):ind.HTLTF(2)),:);
    H=wlanHTLTFChannelEstimate(wlanHTLTFDemodulate(ltf,cfg),cfg);
    data=rx(p+(ind.HTData(1):ind.HTData(2)),:);
    out=wlanHTDataRecover(data,H,nVar,cfg);
    errors=errors+any(out~=bits);
  end
  PER(j)=errors/packets;
  fprintf('SNR %d completed after %d packets, PER: %.5f\n',snr(j),packets,PER(j));
end
figure; semilogy(snr,PER,'-ob'); grid on;
xlabel('SNR [dB]'); ylabel('PER');
title('802.11n 20MHz, MCS15, Direct Mapping, 2x2 Channel Model B-NLOS');
```

% EXP 6: 4x2 IEEE 802.11n TGn channel - result is a spectrum analyzer

```
% Requires WLAN Toolbox and DSP System Toolbox
clc; clear; close all; rng(1);
cfg=wlanHTConfig('NumTransmitAntennas',4, ...
  'NumSpaceTimeStreams',2,'SpatialMapping','Fourier');
bits=randi([0 1],8*cfg.PSDULength,1);
tx=wlanWaveformGenerator(bits,cfg);
ch=wlanTGnChannel('SampleRate',20e6,'NumTransmitAntennas',4, ...
  'NumReceiveAntennas',2,'LargeScaleFadingEffect','None');
rx=ch(tx);
sa=spectrumAnalyzer('SampleRate',20e6,'ShowLegend',true, ...
  'ChannelNames',{'Stream 1','Stream 2'});
sa(rx);
```

% EXP 7: SUI-1 MIMO channel - 2 Doppler PSD and 3 envelope graphs

```
% Requires Communications Toolbox and Signal Processing Toolbox
clc; clear; close all; rng(1);
Fs=0.1e6; N=3e6; % Keep N=3e6 to match record's graph x-axis
ch=comm.MIMOChannel('SampleRate',Fs, ...
  'PathDelays',[0 .4 .9]*1e-6,'AveragePathGains',[0 -15 -20], ...
  'FadingDistribution','Rician','KFactor',4, ...
  'MaximumDopplerShift',.5,'DopplerSpectrum',doppler('Rounded'), ...
  'TransmitCorrelationMatrix',[1 .7;.7 1], ...
  'ReceiveCorrelationMatrix',1,'PathGainsOutputPort',true);
x=pskmod(randi([0 3],N,2),4); [~,g]=ch(x);
fd=ch.MaximumDopplerShift; f=-fd:.01:fd;
a=ch.DopplerSpectrum.Polynomial;
Sd=(a(1)+a(2)*(f/fd).^2+a(3)*(f/fd).^4) ...
  /(2*fd*(a(1)+a(2)/3+a(3)/5))*10^(-15/10);
for k=1:2
  [P,F]=pwelch(g(:,2,k),hamming(N/5),N/10,[],Fs,'centered');
  figure; plot(F/1e3,10*log10(P)); hold on;
  plot(f(Sd>0)/1e3,10*log10(Sd(Sd>0)),'k--');
  axis([-.01 .01 -80 10]); grid on;
  title('Power Spectral Density');
  xlabel('Frequency (kHz)'); ylabel('Power/frequency (dB/Hz)');
  legend('Simulation','Theory');
end
for k=1:3
  figure; semilogy(abs(g(:,k,1)),'b'); hold on;
  semilogy(abs(g(:,k,2)),'r'); grid on;
  title(['Fading envelopes for two transmit links of Path ' num2str(k)]);
  legend('First transmit link','Second transmit link');
end
for k=1:3
  disp(['Tx correlation matrix, path ' num2str(k)]);
  disp(corrcoef(g(:,k,1),g(:,k,2)).');
end
```